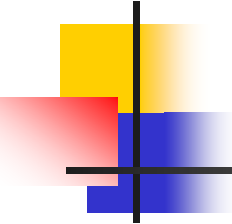


算法设计与分析

Analysis and Design of Algorithm

Lesson 05



要点回顾

- 递归算法
 - Hanoi塔问题
 - 递归优缺点
- 分治策略
 - 概念、适用条件（**最优子结构**）
 - 伪代码、分治法的效率
 - 通过范例学习分治策略的设计技巧
 - 二分搜索
 - 大整数乘法（开始）

大整数的乘法(cont.)

问题：请设计一个有效的算法，可以进行两个 n 位大整数的乘法运算

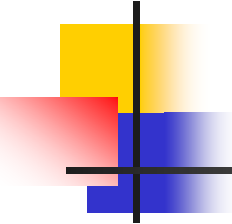
◆小学的方法： $O(n^2)$ **✗ 效率低！！**

◆分治法：

假设： X 和 Y 都是 n 位二进制整数

$$X = \overset{n/2\text{位}}{\boxed{a}} \overset{n/2\text{位}}{\boxed{b}} \quad Y = \overset{n/2\text{位}}{\boxed{c}} \overset{n/2\text{位}}{\boxed{d}}$$

$$\begin{aligned} X &= a 2^{n/2} + b & Y &= c 2^{n/2} + d \\ \rightarrow XY &= ac 2^n + (ad+bc) 2^{n/2} + bd \end{aligned}$$

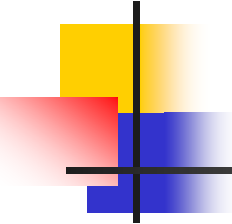


复杂度分析

则计算 XY , 需4次 $n/2$ 位整数乘法(ac 、 ad 、 bc 和 bd)
 需3次小于 $2n$ 位的整数加
 需2次移位($2^n, 2^{n/2}$) } 共需 $O(n)$

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

直接用之前的迭代法解有点点复杂!



换元迭代法

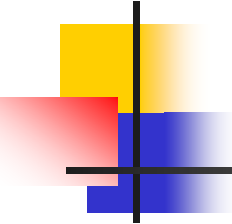
- 将对 n 的递推式换成对其他变元 k 的递推式
- 对 k 直接迭代
- 将解(关于 k 的函数)转换成关于 n 的函数

举例：解如下递推方程

$$\begin{aligned}T(n) &= 2T(n/2) + n - 1 \\ T(1) &= 0\end{aligned}$$

换元：令 $n=2^k$ ，则递推方程变换为

$$\begin{aligned}T(2^k) &= 2T(2^k/2) + 2^k - 1 = 2T(2^{k-1}) + 2^k - 1 \\ T(0) &= 0\end{aligned}$$



换元迭代法

$$T(2^k)=2T(2^{k-1})+2^k-1$$

$$T(0)=0$$

迭代求解：

$$T(n)=T(2^k)=2T(2^{k-1})+2^k-1$$

$$=2[2T(2^{k-2})+2^{k-1}-1]+2^k-1$$

$$=2^2T(2^{k-2})+2^k-2+2^k-1$$

=...

$$=2^kT(1)+k2^k-(2^{k-1}+2^{k-2}+\dots+2+1)$$

$$=k2^k-2^k+1$$

$$=n\log n-n+1$$

最后，需用数学归纳法证明正确性！

复杂度分析

则计算X, Y, 需4次 $n/2$ 位整数乘法(ac, ad, bc和bd)
需3次小于 $2n$ 位的整数加
需2次移位($2^n, 2^{n/2}$) } 共需 $O(n)$

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

为了简化, 设 n 为2的幂



换元迭代法 $n=2^x$

$$T(1) = O(1)$$

$$T(2) = 4O(1) + O(2) \rightarrow 4O(1)$$

$$T(4) = 4(4O(1) + O(2)) + O(4) \rightarrow 4^2 O(1)$$

$$T(8) = 4(4(4O(1) + O(2)) + O(4)) + O(8) \rightarrow 4^3 O(1)$$

复杂度分析

$$T(16) = \dots \rightarrow 4^4 O(1)$$

...

$$\Rightarrow T(n) = O(n^2)$$

$$T(2^x) = \dots \rightarrow 4^x O(1)$$

公式法求解该算法复杂度：

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$

$$\Rightarrow T(n) = O(n^2) \quad \text{✗ 没有改进!!!}$$



这就很尴尬了

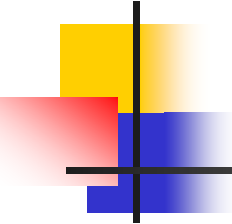
复杂度分析

$$X = \overset{n/2\text{位}}{\boxed{a}} \overset{n/2\text{位}}{\boxed{b}} \quad Y = \overset{n/2\text{位}}{\boxed{c}} \overset{n/2\text{位}}{\boxed{d}}$$

$$\begin{aligned} X &= a 2^{n/2} + b & Y &= c 2^{n/2} + d \\ \rightarrow XY &= ac 2^n + (ad+bc) 2^{n/2} + bd \end{aligned}$$

则计算XY，需4次 $n/2$ 位整数乘法(ac, ad, bc和bd)
需3次小于 $2n$ 位的整数加
需2次移位($2^n, 2^{n/2}$) } 共需 $O(n)$

$$T(n) = \begin{cases} O(1) & n = 1 \\ 4T(n/2) + O(n) & n > 1 \end{cases}$$



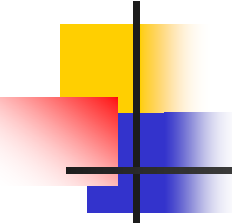
大整数的乘法(cont.)

为了降低时间复杂度，必须减少乘法的次数！！！！

$$1. \quad XY = ac \, 2^n + ((a-b)(d-c) + ac + bd) \, 2^{n/2} + bd$$

$$2. \quad XY = ac \, 2^n + ((a+b)(d+c) - ac - bd) \, 2^{n/2} + bd$$

对于1式：
需3次 $n/2$ 位整数乘法 $((a-b)(d-c), ac, bd)$
需6次加、减
需2次移位



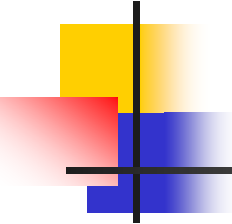
大整数的乘法(cont.)

复杂度分析:

$$T(n) = \begin{cases} O(1) & n = 1 \\ 3T(n/2) + O(n) & n > 1 \end{cases}$$

→ $T(n) = O(n^{\log 3}) = O(n^{1.59})$ ✓ 改进了!!!

细节问题: 两个XY的复杂度都是 $O(n^{\log 3})$, 但考虑到 $a+b$, $c+d$ 可能得到 $n+1$ 位的结果, 使问题的规模变大, 故不选择第2种方案。



大整数的乘法(cont.)

- 还有没有更快的方法？ $T(n)=O(n^{\log 3})=O(n^{1.59})$
- 如果将大整数分成更多段，用更复杂的方式把它们组合起来，将有可能得到更优的算法。
- 最终的，这个思想导致了快速傅利叶变换(Fast Fourier Transform)的产生。该方法也可以看作是一个复杂的分治算法。

year	algorithm	order of growth
?	brute force	$\Theta(n^2)$
1962	Karatsuba-Ofman	$\Theta(n^{1.585})$
1963	Toom-3, Toom-4	$\Theta(n^{1.465}), \Theta(n^{1.404})$
1966	Toom-Cook	$\Theta(n^{1+\varepsilon})$
1971	Schönhage–Strassen	$\Theta(n \log n \log \log n)$
2007	Fürer	$n \log n 2^{O(\log^* n)}$

是否能找到线性时间算法？目前为止还没有结果。

递归树法验证换元迭代法

■ 递归树的概念

- 递归树是迭代计算的模型（迭代的图形表示）
- 递归树的生成过程与迭代过程一致
- 树上所有项恰好是迭代之后产生和式中的项
- 对递归树上的项求和就是迭代后方程的解



递归树法验证换元迭代法

■ 迭代在递归树中的表示

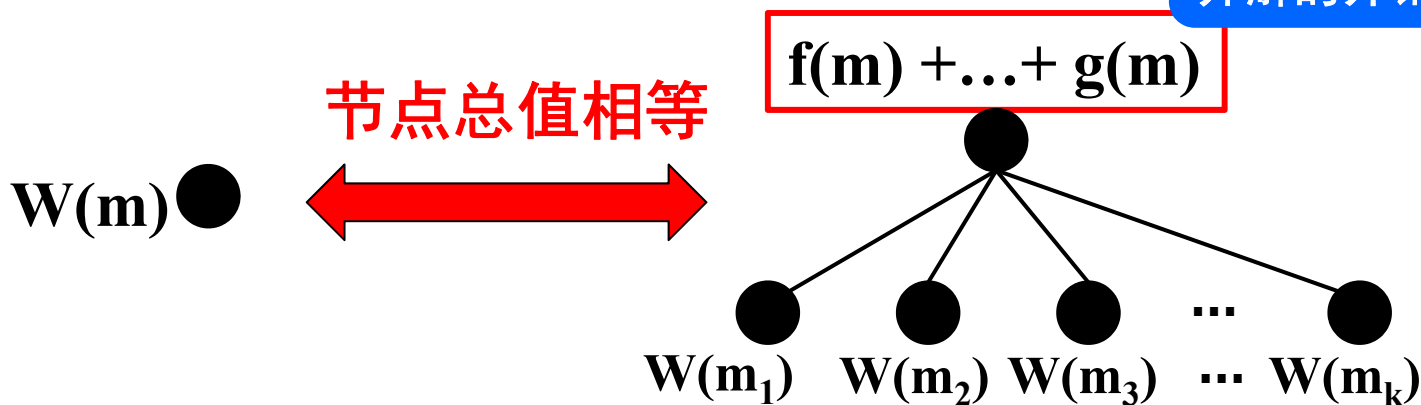
- 如果递归树上某节点标记为 $W(m)$

- $W(m) = W(m_1) + \dots + W(m_k) + f(m) + \dots + g(m), m_1, \dots, m_k < m$

递归求解划分后的子问题的开销

其中 $W(m_1), \dots, W(m_k)$ 称为函数项

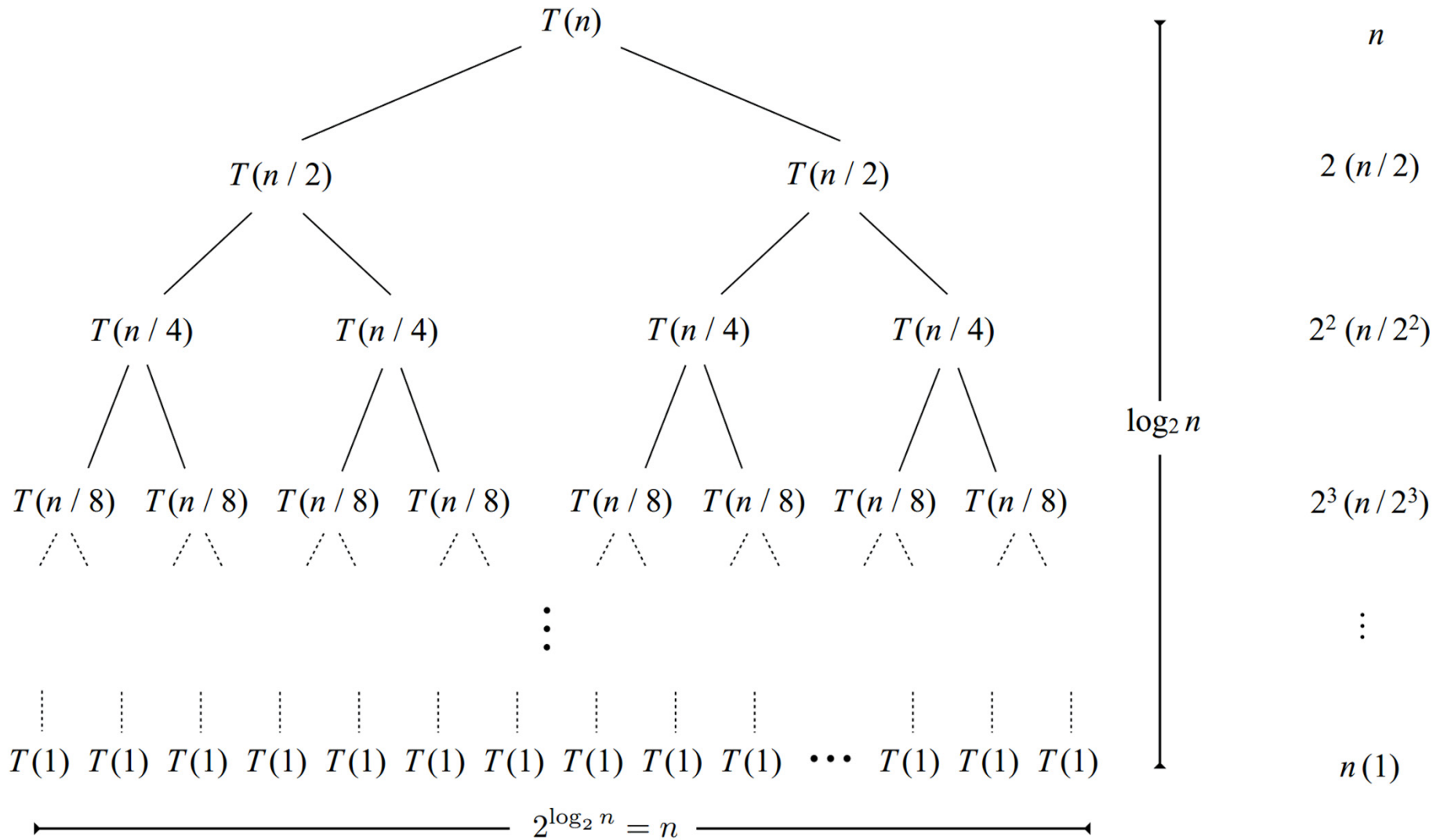
归约到子问题及合并解的开销



每个叶节点是一个函数项

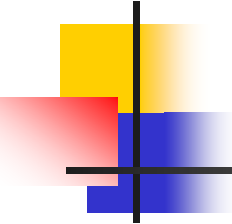
例子： 画出 $T(n)=2T(n/2)+n$ 递归树
 $T(1)=1$

$$T(n) = 2T(n/2) + n, \quad T(1) = 1$$



$$r = 1$$

$$T(n) = (1 + r + r^2 + r^3 + \dots + r^{\log_2 n}) n = n (\log_2 n + 1)$$

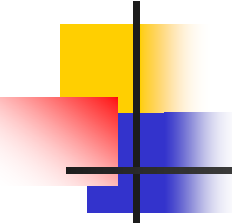


练习

- 画出如下函数的递归树

- $$T(n) = \begin{cases} O(1), & n = 1 \\ T(\frac{1}{3}n) + T(\frac{2}{3}n) + n, & n > 1 \end{cases}$$

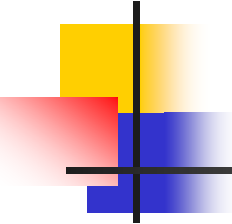
- $$T(n) = T(\alpha n) + T(\beta n) + O(n), \alpha + \beta = 1, \alpha \leq \beta$$



思考

- 如何利用递归树方法估计Fibonacci数列计算的时间复杂度？

$$F(n) = \begin{cases} 1 & , n = 0 \\ 1 & , n = 1 \\ F(n-1) + F(n-2) & , n > 1 \end{cases}$$



回到公式

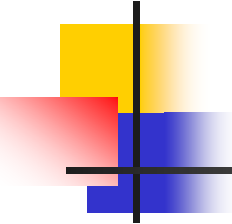
$$T(n) = \begin{cases} O(1) & n = 1 \\ aT(n/b) + f(n) & n > 1 \end{cases}$$

方程的解：

$$T(n) = n^{\log_b a} + \sum_{i=0}^{(\log_b n)-1} a^i f(n/b^i)$$

- 第一项为所有最小子问题的计算工作量
- 第二项为迭代过程归约到子问题及合并解的工作量

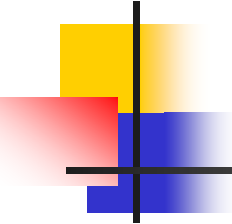
哪一项更主要？



主定理(Master Theorem)

定理： 设 $a \geq 1$, $b > 1$ 为常数, $f(n)$ 为函数, $T(n)$ 为非负数, 且 $T(n) = aT(n/b) + f(n)$, 则:

1. 若 $f(n) = O(n^{(\log_b a) - \varepsilon})$, 存在 $\varepsilon > 0$ 是常数, 则有 $T(n) = \Theta(n^{\log_b a})$
2. 若 $f(n) = \Theta(n^{\log_b a})$, 则有 $T(n) = \Theta(n^{\log_b a} \log n)$
3. 若 $f(n) = \Omega(n^{(\log_b a) + \varepsilon})$, 存在 $\varepsilon > 0$ 是常数, 且对所有充分大的 n 有 $af(\frac{n}{b}) \leq cf(n)$, $c < 1$ 是常数, 则有 $T(n) = \Theta(f(n))$



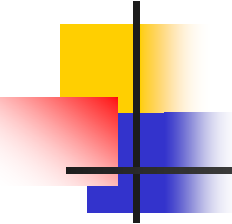
Master定理

■ **例子1:** 求解 $T(n) = 9T\left(\frac{n}{3}\right) + n$

$$a = 9, b = 3, f(n) = n, n^{\log_b a} = n^2$$

$$\because f(n) = n < O(n^{\log_b a}) = n^2 \text{ 取 } \varepsilon = 1 \text{ 即可}$$

$$\therefore T(n) = \Theta(n^{\log_b a}) = \Theta(n^2)$$



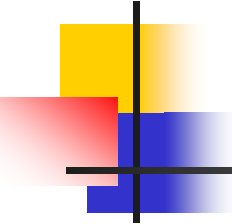
Master定理

■ **例子2:** 求解 $T(n) = T\left(\frac{2n}{3}\right) + 1$

$$a = 1, b = \frac{3}{2}, f(n) = 1, n^{\log_b a} = n^{\log_{3/2} 1} = 1$$

$$\therefore f(n) = 1 = \Theta(n^{\log_b a}),$$

$$\therefore T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$$



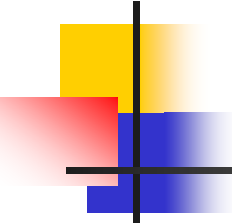
Master定理

- **例子3:** 求解 $T(n) = 3T\left(\frac{n}{4}\right) + n \log n$

$$a = 3, b = 4, f(n) = n \log n, \quad n^{\log_b a} = n^{\log_4 3} \approx n^{0.793}$$

$$f(n) = n \log n = \Omega(n^{\log_4 3 + \varepsilon}) \approx \Omega(n^{0.793 + \varepsilon})$$

取 $\varepsilon = 0.2$ 即可



Master定理

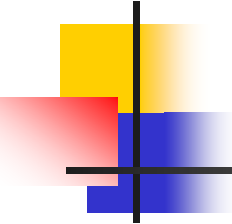
- **例子3:** 求解 $T(n) = 3T\left(\frac{n}{4}\right) + n \log n$

条件验证 $af\left(\frac{n}{b}\right) \leq cf(n)$

$a = 3, b = 4, f(n) = n \log n$, 代入上式

$$3\left(\frac{n}{4}\right) \log\left(\frac{n}{4}\right) \leq c \times n \log n \quad \text{只要 } c \geq 3/4 \text{ 即可}$$

$$\therefore T(n) = \Theta(f(n)) = \Theta(n \log n)$$



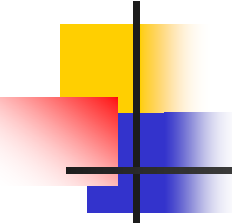
递归算法分析

- 二分检索: $T(n) = T(n / 2) + 1, T(1) = 1$

$$a = 1, b = 2, n^{\log_2 1} = 1, f(n) = 1$$

属于第二种情况

$$T(n) = \Theta(\log n)$$



不能使用主定理的例子

- 例如：求解 $T(n) = 2T(n/2) + n \log n$

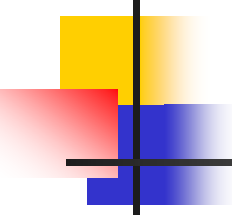
$$a = b = 2, n^{\log_2 2} = n, f(n) = n \log n$$

不存在 $\epsilon > 0$ 使右式成立 $n \log n = \Omega(n^{1+\epsilon})$

不存在 $c < 1$ 使 $af(\frac{n}{b}) \leq cf(n)$ 对所有充分大的 n 成立

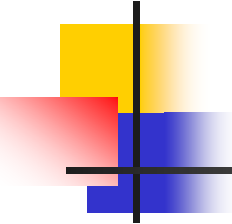
$$2(n/2) \log(n/2) = n(\log n - 1) \leq cn \log n$$

可以考虑递归树！！！！



递推方程求解方法小结

1. 迭代法
2. 换元迭代法
3. 递归树
4. 公式法
5. 主定理



思考

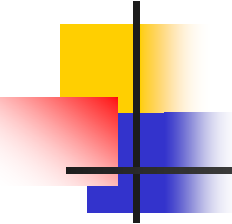
- 正整数划分问题的时间复杂度？

$$q(n, m) = \begin{cases} 1 & n = 1, m = 1 \\ q(n, n) & n < m \\ 1 + q(n, n - 1) & n = m \\ q(n, m - 1) + q(n - m, m) & n > m > 1 \end{cases}$$

图像与矩阵



- 图像有着宽度(width)和高度(height);
- 而矩阵有着行(row)和列(column), 矩阵的操作在数学和计算机中的处理都很常见且成熟;
- 于是很自然的就把图像作为一个矩阵, 把对图像的操作转换成对矩阵的操作。

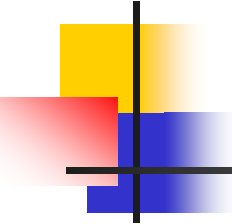


Strassen矩阵乘法

A和B的乘积矩阵C中的元素C[i,j]可定义为:

$$C[i][j] = \sum_{k=1}^n A[i][k]B[k][j]$$

分析：若依此定义来计算A和B的乘积矩阵C，则每计算C的一个元素C[i][j]，需要做n次乘法和n-1次加法。因此，算出矩阵C的n²个元素所需的计算时间为O(n³)



Strassen矩阵乘法

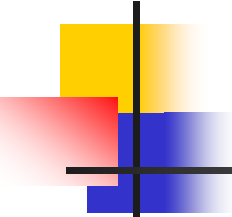
分治法:

使用与大整数相乘类似的技术，将矩阵A，B和C中每一矩阵都分块成4个大小相等的子矩阵。由此可将方程 $C=AB$ 重写为：

$$\begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix} = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

由此可得：

$$\begin{aligned} C_{11} &= A_{11} B_{11} + A_{12} B_{21} \\ C_{12} &= A_{11} B_{12} + A_{12} B_{22} \\ C_{21} &= A_{21} B_{11} + A_{22} B_{21} \\ C_{22} &= A_{21} B_{12} + A_{22} B_{22} \end{aligned}$$



复杂度分析

- 1) $n=2$, 子矩阵阶为1, 8次乘和4次加, 直接求出;
- 2) 子矩阵阶大于2, 为求子矩阵积可继续分块, 直到子矩阵阶降为2。

此想法就产生了一个分治降阶递归算法。

两个 n 阶方阵的积 $\rightarrow$ 8个 $n/2$ 阶方阵积和4个 $n/2$ 阶方阵加。

可在 $O(n^2)$ 时间内完成

计算时间耗费 $T(n) = \begin{cases} O(1) & n = 2 \\ 8T(n/2) + O(n^2) & n > 2 \end{cases}$

所以 $T(n) = O(n^3)$, 与原始定义计算相比并不有效。